

ANPR (Automatic Number Plate Recognition) System

YOLOv8-Based Real-Time Indian License Plate Detection & OCR

Project by: C H Udaya Kumar

GitHub Repository: <https://github.com/UKR555/ANPR->

1. Project Overview

The Automatic Number Plate Recognition (ANPR) System is an end-to-end deep-learning solution designed to automatically detect vehicle license plates and recognize plate numbers in images, videos, and real-time webcam streams.

The system is optimized specifically for Indian vehicle number plate patterns, including:

- Standard formats: MH 12 AB 1234, KA 05 MK 4321
- Compact formats: DL3CAB1234
- Bharat-series numbers and spacing variations

The detection engine uses the YOLOv8 object detection framework, and the text recognition stage uses Tesseract OCR to read the characters from detected plates.

The project supports training custom models, running inference, and extracting number plate text in an automated pipeline.

2. Project Objectives

- Detect license plates accurately in real-time scenarios.
- Recognize and extract text from plates using OCR.
- Support multiple input sources:
 - Webcam streams
 - Video files
 - Static images
- Train a custom YOLOv8 model using an annotated Indian plates dataset.
- Achieve high detection speed and accuracy on both GPU and CPU hardware.

3. System Features

3.1 Detection Capabilities

- Real-time detection using webcam
- Video file processing
- Single image analysis

3.2 Deep Learning Model

- Built using YOLOv8
- Two trained classes:
 - licence – text-focused cropped plate area

- plate – full number plate region

3.3 OCR Integration

- Tesseract OCR is used to read the text detected inside license plates.
- Recognizes various number plate formats common across Indian states.

3.4 Training Workflow

- Fully supports custom training on YOLO-format datasets.
- Automated weight saving and model evaluation.

3.5 Output

- Bounding boxes around detected plates.
- Display of recognized license plate numbers.
- Optional saved result images/videos.

4. Technologies Used

Component	Technology
Programming Language	Python 3.8+
Object Detection Model	YOLOv8 (Ultralytics)
OCR Engine	Tesseract OCR
Dataset Format	YOLO annotation format
GPU Support	NVIDIA CUDA
Camera Input	Laptop / USB Webcam
IDE	VS Code
OS Support	Windows / Linux / macOS

5. Repository Structure

Key files and folders from the project repository:

ANPR/

- | — README.md → Project documentation
- | — requirements.txt → Python dependency list
- | — running_ANPR.py → Main pipeline runner
- | — simple_anpr.py → Lightweight detection & OCR script
- | — new.py → Alternative inference workflow
- | — anpr_train.py → Model training script
- | — test_model.py → Model evaluation script
- | — utils.py → Helper functions
- | — prebooked_plates.txt → Sample dataset entries
- | — yolov8n.pt / yolov8s.pt → Pretrained YOLOv8 models
- | — model/anpr_model/ → Trained model outputs
- | — Dataset/
 - | — train/images + labels
 - | — valid/images + labels
 - | — test/images + labels

6. Dataset Format

The project uses the standard YOLO dataset structure:

Dataset/

- | — train/
 - | — images/
 - | — labels/
- | — valid/
 - | — images/
 - | — labels/
- | — test/
 - | — images/
 - | — labels/

Each label file follows the YOLO format:

<class_id> <x_center> <y_center> <width> <height>

The dataset annotations are customized to cover:

- Variations in plate spacing
- Different state codes
- Bharat-series formats

7. Installation & Setup

Step 1 – Clone the repository

```
git clone https://github.com/UKR555/ANPR-  
cd ANPR-
```

Step 2 – Install dependencies

```
pip install -r requirements.txt
```

Step 3 – Install Tesseract OCR

- Windows: Download installer and add path to environment variable
- Linux:

```
sudo apt install tesseract-ocr
```

- macOS:

```
brew install tesseract
```

Step 4 – Configure OCR Path

If required, add the Tesseract path inside:

```
anpr_ocr.py
```

8. Model Training

To train the YOLOv8 ANPR model:

```
python anpr_train.py
```

After training, the weights are saved to:

```
runs/detect/anpr_model/weights/
```

9. Detection & Inference

9.1 Webcam (Real-Time)

```
python anpr_detect_realtime.py
```

Press Q to exit.

9.2 Video Detection

```
python anpr_detect_video.py --video video.mp4
```

Optional arguments:

- --model path/to/model.pt
- --output output.mp4

- --no-display
- --no-save

9.3 Image Detection

`python anpr_image_detection.py --image input.jpg`

Options:

- --conf 0.25
- --output-dir results
- --no-display
- --no-save

9.4 OCR (Text Extraction)

`python anpr_ocr.py --image input.jpg`

Options:

- --conf 0.4
- --output-dir ocr_results
- --no-display

10. Performance

Hardware	FPS (Approx.)
GPU (CUDA)	20 – 40 FPS
CPU only	1 – 5 FPS

Performance depends on system specifications.

11. Use Cases

Automated traffic monitoring
Toll booth verification
Smart parking management
Law enforcement vehicle tracking
Campus or society vehicle access control

12. Strengths of the Project

- Complete end-to-end ANPR pipeline
- Indian license plate format optimization
- Real-time processing capability
- Modular training and testing setup
- Lightweight YOLOv8 models for fast inference

13. Future Enhancements

- Accuracy improvement using Vision Transformers
- Multi-camera tracking support
- Integration with cloud databases
- Mobile application support
- Advanced OCR models for low-light & motion blur environments

14. Licensing

- Dataset and related materials are licensed under Creative Commons CC BY 4.0
- Code is provided for educational and research purposes.

15. Conclusion

This project demonstrates a complete working solution for Automatic Number Plate Recognition using YOLOv8 and OCR, tailored specifically for Indian vehicle plates. The system is capable of detecting plates in real time and extracting text with high speed and reasonable accuracy.

It provides an excellent foundation for real-world intelligent transportation systems and showcases applied skills in computer vision, deep learning, dataset annotation, model training, and deployment.